



Material de Apoio — Laços de Repetição em Python (com C e R como contraponto)

Prof. Bruno Melo · Universidade Positivo — Processo Seletivo Docente

Conteúdo complementar (fora dos 20 min) · guia de consulta

Sumário

1	O que é uma estrutura de repetição	2
2	O exemplo condutor: simulador de poupança	2
2.1	Em Python (DOWNEY, 2016) — linguagem principal	2
2.2	Em C (KERNIGHAN; RITCHIE, 1989) — contraponto	3
2.3	Em R (WICKHAM; GROLEMUND, 2017) — contraponto	3
3	Quadro comparativo	3
4	Erros comuns (para a depuração ao vivo)	4
5	Pergunta de verificação de aprendizagem	4
6	Referências	4

A ideia central: a estrutura lógica de um laço — iniciar, testar/percorrer, executar, atualizar — é a **mesma** em Python, C e R. Muda a *sintaxe* e o que cada comunidade considera idiomático. Aqui, **Python está em foco**; C e R aparecem apenas como contraponto.

1 O que é uma estrutura de repetição

Uma **estrutura de repetição** (laço, *loop*) executa um bloco de comandos várias vezes enquanto uma condição se mantém verdadeira ou enquanto se percorre uma sequência de valores (DOWNEY, 2016). Distinguímos dois casos:

- **Repetição contada** — o número de iterações é **conhecido** de antemão (em Python, um `for` sobre um `range`).
- **Repetição condicionada** — o número de iterações **depende de uma condição** avaliada em tempo de execução (em Python, um `while`) (ASCENCIO; CAMPOS, 2012).

Todo laço tem, em essência, quatro elementos: **início**, **condição/percurso**, **corpo** e **atualização** da variável de controle. A omissão da atualização da variável testada é a causa clássica do **laço infinito** (DOWNEY, 2016).

2 O exemplo condutor: simulador de poupança

Problema: o usuário guarda um valor fixo por mês. Quantos meses até atingir uma meta (ex.: comprar um notebook)? E quanto terá em exatamente 12 meses?

2.1 Em Python (DOWNEY, 2016) — linguagem principal

```
# Contada: percorre um intervalo de 12 meses (o for idiomático em Python)
total = 0.0
for mes in range(1, 13):          # 1, 2, ..., 12 (o 13 fica de fora)
    total += mensal
    print(f"Mes {mes}: R$ {total:.2f}")

# Condicionada: repete até a meta
total, meses = 0.0, 0
while total < meta:
    total += mensal
    meses += 1

# "Ao menos uma vez" (Python não tem do-while): while True + break
total, meses = 0.0, 0
while True:
    total += mensal
    meses += 1
    if total >= meta:
        break
```

Em Python, o `for` percorre os elementos de uma sequência (aqui, `range`) e não um contador manual: iterar sobre coleções é o estilo idiomático (DOWNEY, 2016). Não existe `do-while`; o padrão equivalente é `while True:` com `break` no fim.

2.2 Em C (KERNIGHAN; RITCHIE, 1989) — contraponto

```

/* Contada: 12 meses fixos */
float total = 0.0f;
for (int mes = 1; mes <= 12; mes++) {
    total += mensal;
    printf("Mes %d: R$ %.2f\n", mes, total);
}

/* Condicionada: ate a meta */
float t = 0.0f; int meses = 0;
while (t < meta) { t += mensal; meses++; }

/* "Ao menos uma vez": do-while (existe em C) */
do { t += mensal; meses++; } while (t < meta);

```

2.3 Em R (WICKHAM; GROLEMUND, 2017) — contraponto

```

# Contada
total <- 0
for (mes in 1:12) {
    total <- total + mensal
    cat(sprintf("Mes %d: R$ %.2f\n", mes, total))
}

# Condicionada
t <- 0; meses <- 0
while (t < meta) { t <- t + mensal; meses <- meses + 1 }

# Estilo idiomático em R: vetorizacao (evita o laço explicito)
total_12 <- sum(rep(mensal, 12))

```

Em R, laços explícitos funcionam, mas o estilo idiomático prefere **vetorização** e funções de iteração funcional (ex.: `purrr::map`), reservando o `for` para efeitos colaterais (WICKHAM; GROLEMUND, 2017, cap. *Iteration*).

3 Quadro comparativo

Aspecto	Python	C	R
for contado	for m in range(1,13):	for(int m=1;m<=12;m++)	for (m in 1:12)
while	while cond:	while(cond){}	while (cond) {}

Aspecto	Python	C	R
“ao menos 1 vez”	<code>while True: + break</code>	<code>do{}while(cond);</code>	<code>repeat{} + break</code>
Bloco	indentação após <code>:</code>	chaves <code>{ }</code>	chaves <code>{ }</code>
Controle	nome dinâmico	tipada, declarada	nome dinâmico
Idiomático	iterar coleções	contador explícito	vetorização / <code>map</code>

Conclusão didática: a estrutura lógica (iniciar, testar/percorrer, executar, atualizar) é a mesma nas três linguagens; muda a sintaxe e o que cada comunidade considera idiomático. Dominar o conceito em Python facilita a leitura de C e R.

4 Erros comuns (para a depuração ao vivo)

1. **Laço infinito** — a variável testada no `while` não é atualizada no corpo. *Ex.:* esquecer `total += mensal`; `total` fica 0 e a condição nunca fica falsa (DOWNEY, 2016).
2. **Off-by-one** — erro de contagem por um: usar `range(1, 12)` quando se quer 12 iterações gera só 11, porque `range` **exclui o fim**.
3. **Sintaxe do bloco** — esquecer os dois-pontos `:` no cabeçalho do laço, ou misturar espaços e tabs, provocando `IndentationError` (armadilha típica de Python).

5 Pergunta de verificação de aprendizagem

No simulador de poupança, quero garantir que o menu apareça pelo menos uma vez e só depois perguntar se o usuário deseja continuar. Como fazer isso em Python — e por que o `while` comum, sozinho, não é a forma mais direta?

Resposta esperada: usar `while True:` com um `break` **no fim** do corpo. Assim o corpo (o menu) executa **pelo menos uma vez** antes do primeiro teste — o padrão idiomático que substitui o `do-while`, inexistente em Python (DOWNEY, 2016).

Questão-desafio (opcional para a banca): “Em C, qual estrutura resolveria isso diretamente?” Resposta: o `do-while`, que testa a condição ao final (KERNIGHAN; RITCHIE, 1989).

6 Referências

- ASCENCIO, A. F. G.; CAMPOS, E. A. V. **Fundamentos da programação de computadores:** algoritmos, Pascal, C/C++ (padrão ANSI) e Java. 3. ed. São Paulo: Pearson, 2012.
- DOWNEY, A. B. **Pense em Python:** pense como um cientista da computação. 2. ed. São Paulo: Novatec, 2016.
- KERNIGHAN, B. W.; RITCHIE, D. M. **C: a linguagem de programação:** padrão ANSI. Rio de Janeiro: Campus, 1989.
- MASETTO, M. T. **Competência pedagógica do professor universitário.** 2. ed. São Paulo: Summus, 2012.

WICKHAM, H.; GROLEMUND, G. **R for data science**: import, tidy, transform, visualize, and model data. Sebastopol: O'Reilly Media, 2017.